

CTAS

#DB Table

Table is a structured collection of data, similar to a spreadsheet or grid (each).

TABLE TYPES

PERMANENT

TEMPORARY

CREATE/SELECT CTAS

1. CREATE/define the structure of Table.
2. INSERT/insert data into the Table.

CTAS: CREATE TABLE AS SELECT

Create a new table based on the result of a SQL query.

- Query of View has not yet executed.
- Query of CTAS has been executed already.

• Querying View is slower than Querying CTAS Tables.

- Query of View give fresh data from original Table.

- Query of CTAS has old data not from original it already fetch data from original and save it.

~~CTAS~~ Syntax: CREATE/INSERT

CREATE TABLE Table_name

DDL Statement. $\leftarrow$ $\begin{array}{l} \text{C ID INT} \\ \text{NAME VARCHAR(10)} \\ \end{array}$

Insert Statement. ← INSERT INTO Table-name
VALUES (1, 'Frank')

CTAS Syntax.

CREATE TABLE NAME AS → DDL Statement

(
SELECT
FROM
WHERE
) → Query

MySQL | PostgreSQL | Oracle.

SELECT
INTO New-Table
FROM ..
WHERE..
SQL server (MS)

use case 1: optimize performance.

use case 2: creating a snapshot.

↓
Make a copy of that
Table and use it.

use case 3: physical data ~~read~~ Marks in DWH.
persisting the data marks of a DWH. Improves
the speed of data retrieval compared to using
view.

Temporary Tables :- Stores intermediate result in
temporary with the database during the session.
• The database will drop all temporary tables
once the session ends.

* SESSION: Time between connecting to and disconnecting from the database.

Syntax & Example

SELECT ---

INTO (#)NEW-table.

FROM --- → Temp table.

WHERE ---

How DB Executes Temp Tables.

Intermediate Result

STORED PROCEDURE

- What is stored procedure?
predefined set of SQL statements that are stored in and executed on the database server.
- Function in SQL that performs a specific task (like insert, update, delete etc) and can be reused any time.

CREATE PROCEDURE procedureName AS

BEGIN

-- SQL statements go here

END

} explanation.

EXEC procedureName [@parameter_name]

STORED PROCEDURE PARAMETER.

Placeholders used to pass value as input from caller to the procedure, allowing dynamic data to be processed.

* Avoid Repetition: If you notice repeated code in your project, it's sign that your code can be improved.

Ex: CREATE PROCEDURE CustomerSummary @Country NVARCHAR(50)
AS
BEGIN

↓
parameter.

SELECT
COUNT(*) TotalCustomers,
AVG(score) avgScore
FROM sales.Customers
WHERE Country = @Country
END

Execution: EXEC CustomerSummary @Country = 'USA'

add default value.

CREATE PROCEDURE CustomerDetail @Country NVARCHAR(50)
= 'default USA'
AS

stored procedure (multiple statements).

Variables: placeholders used to store values to be used later in the procedure.

CREATE PROCEDURE GetCustomerSummary @Country
NVARCHAR(50) = 'USA'

AS

BEGIN

DECLARE @TotalCustomers INT, @AvgScore FLOAT;
SELECT

@Totalcustomers = COUNT(*)

@AvgScore = AVG(Score)

FROM sales.customers

WHERE country = @country

END

4 STORED PROCEDURE

CONTROL FLOW : IF ELSE

ALTER PROCEDURE OvercustomersSummary @country
NVARCHAR(50) = 'USA'

AS

BEGIN

DECLARE @Totalcustomers INT, @AvgScore FLOAT

-- prepare & cleanup

IF EXISTS (SELECT 1 FROM sales.customers WHERE score
IS NULL AND COUNTRY = @country)

BEGIN

PRINT('Updating NULL score to 0')

UPDATE sales.customers

SET score = 0

WHERE score IS NULL AND COUNTRY = @country

END

ELSE

BEGIN

PRINT('NO NULL scores found')

END

END

ERROR HANDLING (TRY CATCH)

• Syntax.

BEGIN TRY

-- SQL statement that might cause an error

END TRY

BEGIN CATCH

-- SQL statements to handle the error

TRY CATCH

Built in function for Error

PRINTC('An error occurred---')

PRINTC('Error message : ' + ERROR_MESSAGE());

PRINTC('Error Number : ' + CAST(ERROR_NUMBER() AS
NVARCHAR));

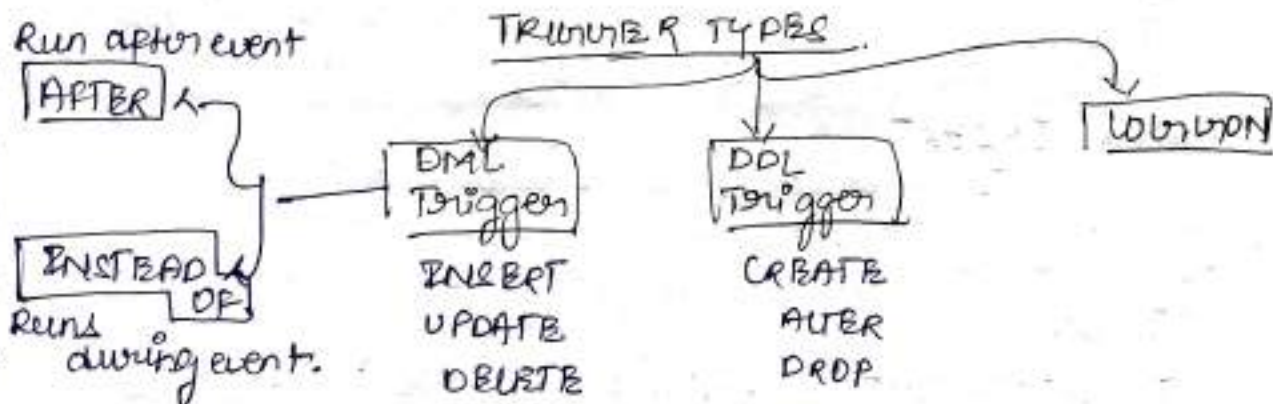
ERROR_LINE;

ERROR_PROCEDURE;

~~# Triggers~~

2. TRIGGERS

Special stored procedure (set of statements) that automatically runs in response to a specific event on a table or view.



Triggers use case
WORKING.

Trigger Syntax.

```
CREATE TRIGGER triggername ON tablename.  
WHEN → AFTER INSERT, UPDATE, DELETE  
BEGIN  
WHAT -- SQL statement  
END
```

#INSERTED: Virtual Table that holds a copy of the rows that are being inserted into the target table.

-- Step 1 Create Table.

```
CREATE TABLE Sales.EmployeeLogs (  
    LogID INT IDENTITY(1,1) PRIMARY KEY,  
    EmployeeID INT,  
    LogMessage VARCHAR(255),  
    LogDate DATE
```

-- Step 2 Create Triggers.

```
CREATE TRIGGER trg-afterinsert ON Sales.Employees  
This will create a trigger named trg-afterinsert  
on table Sales.Employees
```

```
AFTER INSERT
```

Trigger will execute after a new row is inserted.

```
AS
```

```
BEGIN
```

```
INSERTED INTO Sales.EmployeeLogs (EmployeeID,  
    LogMessage, LogDate)
```

• Means are inserting data into the Table Sales.EmployeeLogs and we will fill 3 columns.

SELECT EmployeeID,

'New Employee added = ' + CAST(EmployeeID AS NVARCHAR),

GET DATE()

FROM INSERTED

↳ gets newly inserted employeeID from inserted table.

↳ create text message that includes text.

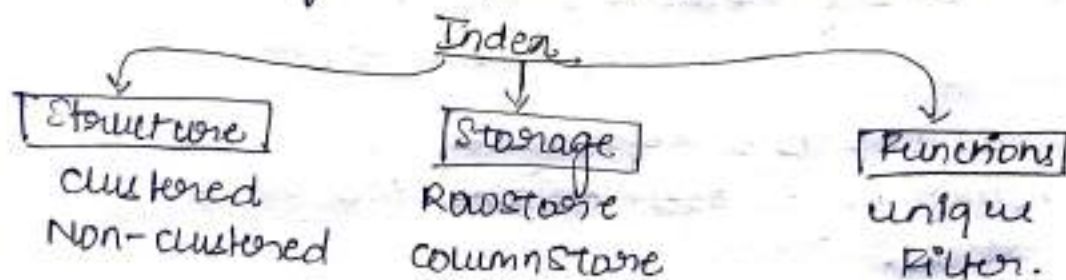
↳ get current system date and time when the trigger runs.

↳ reads the data of newly inserted rows (the special temporary table created by SQL server during the insert).

ENDBLOCK

• what is an index?

data structure provides quick access to data, optimizing the speed of your queries.



Trade-off: some indexes are better for reading, others for writing performance.

Page: Smallest unit of data storage in a database (8kb).

• It stores anything (data, metadata, indexes, etc).

Types: - Data Page.

- Index page.

• Heap: Heap is a table without any clustered index. Means data rows are stored randomly - not in order.

Heap = unordered storage (no index)

when cluster index created no heap is no longer.

Data page: Data page is a actual storage page in SQL server where rows of table are stored.

- SQL server stores data in 8 kb pages
- table data is kept beside these data pages.
- when you create index, index points to these data pages. so SQL server can find rows faster.

Clustered Index: Clustered Index decides the physical order of the data in a table.

- arranges the data rows in sorted order based on the index key.
- so the table itself becomes ordered.
- Only one clustered index can exist per table.

B+ : Hierarchical structure storing data at leaves, to help quickly locate data.

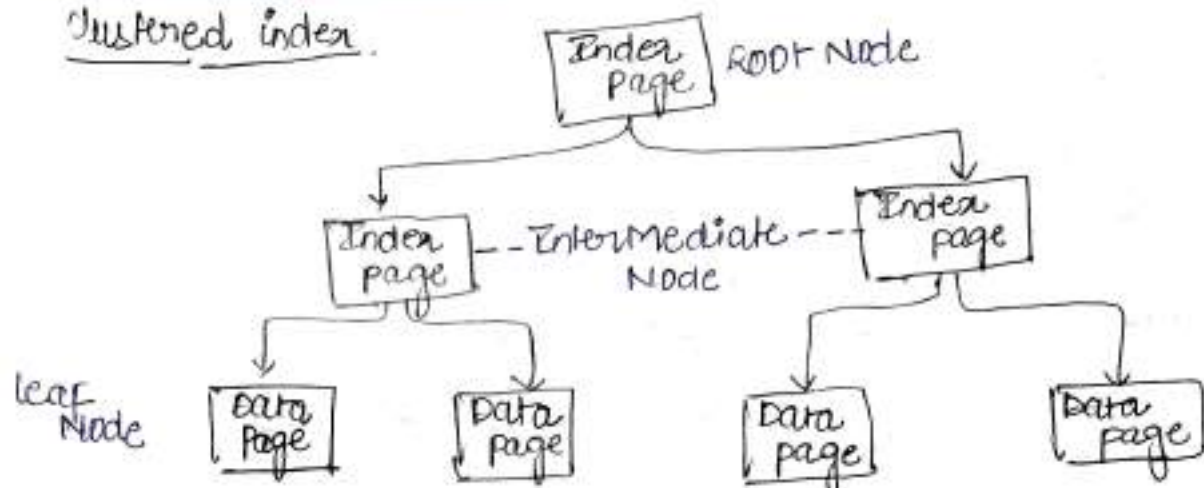
INDEX PAGE: It stores key values (pointers) to another page. doesn't store the actual rows.

- Root Node: Starting point.
- Intermediate Node: help in navigation.
- leaf Nodes: contain actual index pointer (can data for clustered index)

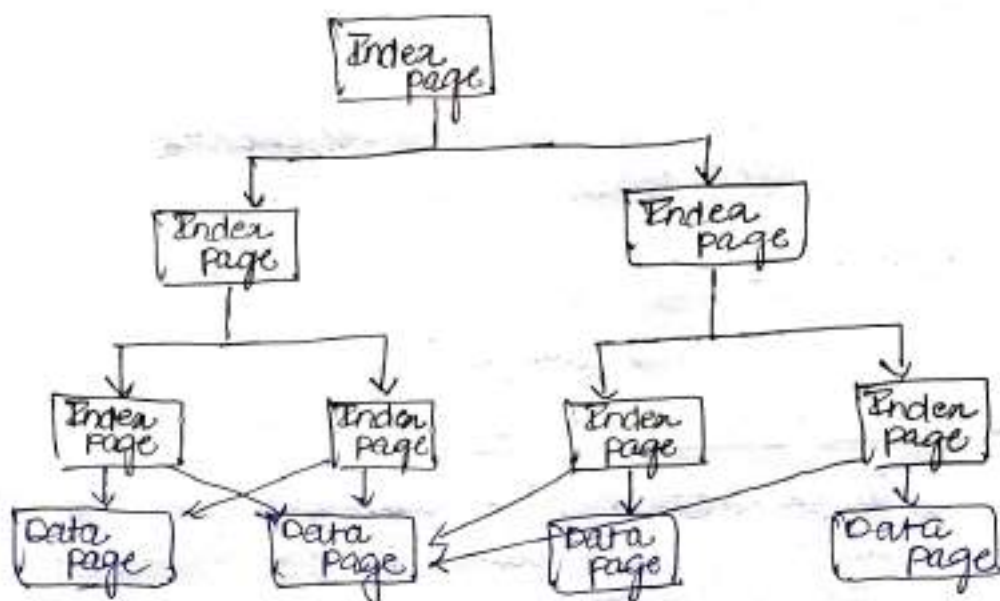
Non-clustered Index: is like an extra index or shortcut created separately from actual table data.

- It does not change the physical order of table rows.
- It stores index key + pointers that point to the data pages where the actual data is stored.
- A table can have multiple non-clustered index.

Clustered Index



Non-clustered Index



Clustered Index

- Physically sorts and stores rows
- One index per table.
- Faster.
- Slower, due to potential data row reordering.
- More storage efficient
- unique column.
- Not frequently modified column.
- Improve Range Query performance.

Non-clustered Index

- Separate structure with pointers to the data.
- Multiple indexes are allowed.
- Slower
- Faster, since physical data order is unaltered.
- ~~Fast~~ Requires additional storage space.
- Columns frequently used in search condition and joins.
- Exact Match queries.

Index Syntax

CREATE [CLUSTERED | NONCLUSTERED] INDEX index-name ON
table-name (col1, col2, ---)
Default is Non-clustered.

- 1) CREATE CLUSTERED INDEX IX-customer-ID ON customers(ID)
- 2) CREATE NONCLUSTERED INDEX IX-customer-city ON customers(CITY)
- 3) CREATE INDEX IX-customer-name ON customers (LastName ASC, FirstName DESC)

1 primary key automatically creates a clustered index by default.

Composite Index

Two columns in index.

Leftmost prefix Rule

Index works only if your query filters start from the first column in the index and follow its order.

Column Store Index

- Stores data column-wise instead of row-wise.

Row-store Index

- Organizes and stores row by row.
- Less efficient in storage.
- Fast speed per read & write operations.
- Higher retrieves all columns

Column-store Index

- Organizes and stores data Column by column.
- Highly efficient with compression.
- Fast read and slow write performance.
- Lower retrieves specific column.

- OLTP: commerce banking, financial system, order processing.
- High frequency transaction applications.
- Quick access to complete record.
- OLAP: data warehouse, BI reporting analytics.
- Big data analytics.
- Scanning of large datasets.
- Fast aggregation.

Syntax.

CREATE [CLUSTERED | NONCLUSTERED] [COLUMNSTORE]
INDEX index-name ON table-name (col1, col2...)

- CREATE NONCLUSTERED COLUMNSTORE INDEX ix_customers -
country ON customers (country)
- CREATE CLUSTERED ~~INDEX~~ COLUMNSTORE INDEX ix_customers
ON customers
↓
NOT allowed to use
columns.

Rule: You can't specify columns in clustered index
columnstore.

Unique Index.

Ensures no duplicate value exists in specific column.

Benefits

- 1). Ensure uniqueness.
- 2). Slightly increase query performance.

Performance

- Writing to an ~~index~~ unique index is slower than non-unique.
- Reading from an unique index is faster than non-unique.

Syntax: ↓ default is NOT unique.

CREATE [UNIQUE] [CLUSTERED] [NONCLUSTERED] INDEX
idx_name ON table-name (col1, col2...)

Index
allow
duplicates
CREATE INDEX idx_customers_email ON customers
(email)

Filtered Index

A index that includes only rows meeting the specified conditions.

Syntax

```
CREATE [UNIQUE] [NONCLUSTERED] INDEX  
index-name ON table-name (col1, col2, ...)   
WHERE [condition].
```

Rules

- You can't create a filtered index on a clustered index.
- You cannot create a filtered index on a columnstore index.

Index Management & Monitoring.

- * Monitor Index usage.
- * Monitor Missing values.
- * Monitor duplicate Indexes.
- * Update statistics.
- * Monitoring fragmentation.

- unused spaces in data pages
- data pages are out of order.

Fragmentation Methods.

• Recognize:

- defragments leaf nodes to keep them sorted - "light" operation.

• Rebuild:

- Recreates Index from scratch.
- "Heavy" operation.

avg-fragmentation-in-percent.
Indicate how out-of-order pages are within the index.

EXECUTION PLAN

- Roadmap generated by a database on how it will execute your query step-by-step.

Execution Plan Basics.

* Estimated Execution Plan.

- Predicts the execution plan without actually running the query.

* Actual Execution plan.

- Shows the execution plan as it occurred after running the query.

* Live Execution plan.

- Shows the real-time execution flow as the query runs.

Estimates vs Actual plan.

If predictions don't match the actual Execution plan, this indicates issues like inaccurate statistics or outdated indexes leading to poor performance.

Left to Right scan.

* Table scan.

- Reads the entire table, page by page and row by row "everything" which can lead to slower query performance on large tables.

* Index scan

Scans all data in an index to find matching rows.

* Execution Plan.

Heap vs clustered

- it should be sorted
- More CPU
- it is already sorted
- less CPU
- Heap is slower than clustered, because database must perform extra work to sort rows.

Tip: After creating new index, check the execution plan to see if your query uses the index.

* Execution plan.

Non-clustered

* Index seek.

A targeted search within an index retrieving only specific rows.

Types of scan

- Table Scan: Reads every row in a table.
- Index Scan: Reads all entries in an index to find the results.
- Index Seek: Quickly locates specific rows in an index.

* Execution plan.

Rowstore vs columnstore

→ are more efficient.

* SQL hints.

commands you add to a query to force the database to run it in a specific way for better performance.

TIPS | SQL HINTS

- Test hints in all projects environments (DEV, PROD) as performance vary.
- Hints are quick fixes (workaround not solution) you still have to find the cause and fix it.

Indexing Strategy

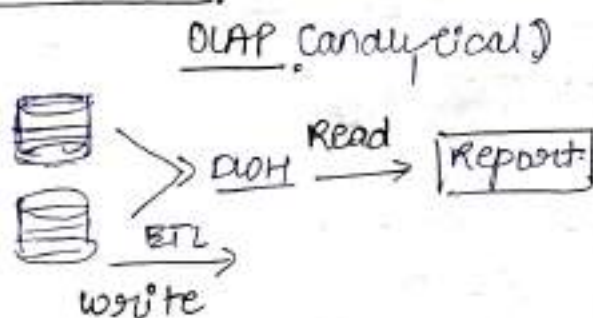
* Avoid over Indexing.

• Indexes slow down write performance, when data is inserted, updated, or deleted, database has to update indexes.

• Too many Indexes can confuse Execution Plan.

#1 phase: Initial strategy

Main goal



Goal: optimize write performance

• columnstore Index

#2 phase: usage patterns.

- Identify frequent used Table & columns.
- choose Right Index.
- Test Index

#3 phase: Scenario-Based-Indexing.

- Identify slow queries.
- Check Execution plan.
- choose right Index.
- (test) compare Execution plans.

#4 phase: Monitoring & Maintenance

- Monitor Index usage.
- Monitor Missing ~~with~~ Indexes.
- Monitor duplicate Indexes.
- update statistics.
- Monitor fragmentations.

TABLE PARTITIONING

#SQL Partitioning

divide big table into smaller partitions.
while still being treated as a single logical table.

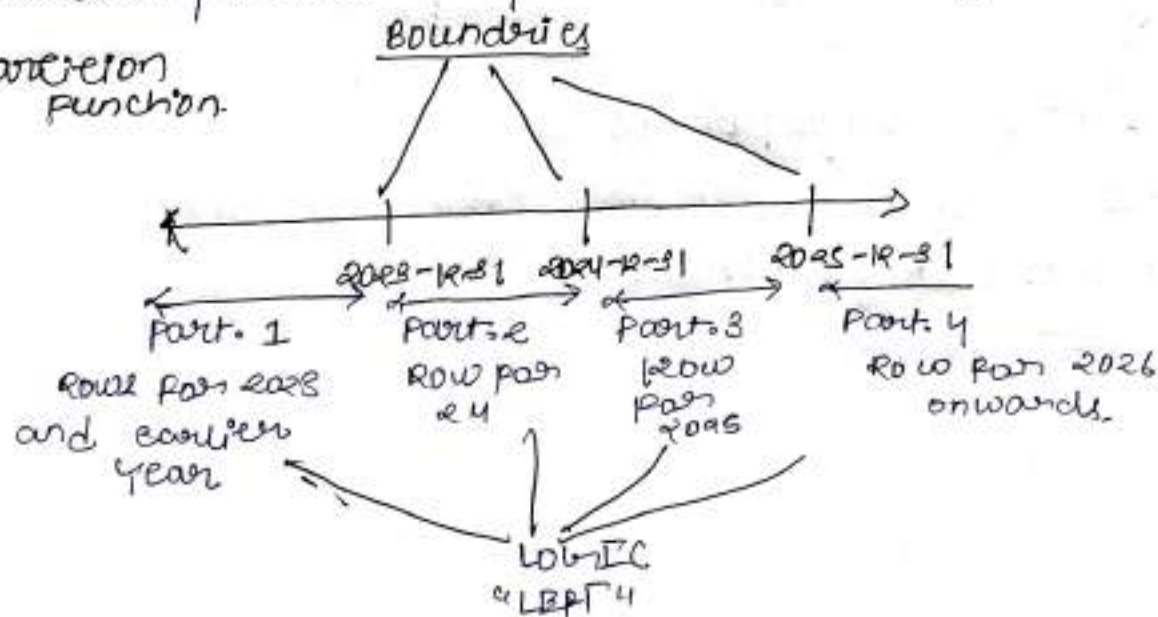
#Building partitions

#1 Create partition function.

- partition function: define the logic on how to divide your data into partitions.

Based on partition key like (Column, Region (date)).

① partition function.



#2 Create File Groups.

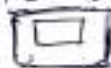
Ex:

Building partitions.

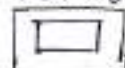
- Filegroups: logical container of one or more data files to help organize partitions.

Filegroups.

FG1-2023



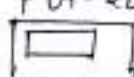
FG1-2024



FG1-2025



FG1-2026



-- primary: default filegroup where all objects of a database is stored.

#3 Create Data Files.

↓
actual data. Each file group like one or more multiple data files. file format is "ndf".

#4 Create Partition schema.

#5 Create Partitioned Tables.